

Experiment 6

AI-Assisted Programming and Debugging

Submission PDF

Source folder on GitHub

Prepared 2026-09-29 from the experiment materials on branch
`codex/complete-experiments`.

Source: [Ex06_AI_Assisted_Programming_Debugging/README.md](#)

Exp-6: AI-Assisted Programming and Debugging

Due: 25 Sep 2026 Reference repo: RaajaThilahar/Ex.No.6

Objective

Design prompts that get AI tools to write, debug, optimise, explain and test code in **Python, C and Java**, then compare manual vs AI-assisted coding.

Task used: Binary search + intentionally buggy version

Folder	Contents
python/	binary_search_buggy.py, binary_search_fixed.py, test_binary_search.py
c/	binary_search_buggy.c, binary_search_fixed.c, test_binary_search.c
java/	BinarySearchBuggy.java, BinarySearch.java, BinarySearchTest.java
prompts/	Prompts for generate / debug / optimise / complexity / unit tests
bugs.md	Bug list found (with how)
code_quality_analysis.md	Deliverable: manual vs AI comparison

Run

```
python3 python/test_binary_search.py
gcc c/test_binary_search.c c/binary_search_fixed.c -o c/t && ./c/t
cd java && javac BinarySearch.java BinarySearchTest.java && java BinarySearchTest
```

Recorded run

Python passed locally (5 tests). C (6 checks) and Java (6 checks) compiled and passed in OneCompiler online on 2026-09-29. For the browser compiler, each implementation and its test cases were combined into one source file because the free editor uses a single-file harness. The original separate-file GCC/JDK commands above were not run locally; see validation_run.md.

Source:

[Ex06_AI_Assisted_Programming_Debugging/LAB_RECORD.md](#)

Lab Record: Exp-6: AI-Assisted Programming and Debugging

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Use AI to write, debug, optimise, explain and test code in Python, C and Java.

Tools / Apparatus

Python: 5 tests passed locally. C: 6 checks passed in OneCompiler online. Java: 6 checks passed in OneCompiler online. Local GCC/JDK are not installed.

Procedure

Prompts written (see prompts/ or the folder README).

Each prompt run on every tool; raw outputs saved under outputs/ (or the folder's equivalent).

Results scored in code_quality_analysis.md on: Bugs found, correctness, complexity, tests.

Findings summarised below.

Observations

See code_quality_analysis.md. Add your own scores for any tool you ran yourself. Tools not run are left blank; do not fill them in from memory.

Result

Buggy and fixed binary search examples, prompts and a code-quality analysis cover Python, C and Java. Python 5/5 passed locally. Combined C implementation and six checks compiled and passed in OneCompiler; combined Java implementation and six checks compiled and passed there. Details are in validation_run.md.

Conclusion

Executing tests exposed errors in earlier bug notes and confirmed all three fixed implementations. C and Java were compiled as single-file test harnesses in OneCompiler; the original separate-file projects still need a local GCC/JDK run if the instructor requires those exact command-line builds.

Faculty signature: _____

Source: [Ex06_AI_Assisted_Programming_Debugging/bugs.md](#)

Bugs found

All buggy files were **executed** (python3, gcc, javac/java; hang checks under timeout). "Found by" is stated honestly: the bugs are annotated in the source comments (they were planted), so what was verified here is which ones actually manifest and on which inputs. Verification was done by AI (Claude) running the code; no manual/human debugging session was recorded.

Environment: python3, gcc, javac/java as installed in this sandbox. Exhaustive check in Python: all sorted arrays $[1, 3, \dots, 2n-1]$, $n = 0..5$, every target $0..2n$ (34 cases).

#	Lang	Bug	Manifests?	Observed failing input	Found by	Fix
1	Python	<code>hi = len(arr)</code> (should be <code>len(arr)-1</code>)	Only in combination. Alone it raises <code>IndexError</code> (tested in isolation, e.g. <code>[]</code> , target 0). In the buggy file it is masked by bug 2 (<code>lo < hi</code> keeps <code>mid < len</code>), so no <code>IndexError</code> was ever seen in the shipped buggy code	none on its own in the shipped file	AI, by running an isolated variant; the claim in the earlier table (" <code>[5]</code> , <code>5</code> ") was wrong	<code>hi = len(arr) - 1</code>
2	Python	<code>lo < hi</code> skips the last candidate	Yes, wrong result: <code>[1, 3]</code> target 1 gives -1 (expected 0), <code>[1, 3, 5]</code> target 1 gives -1. <code>[5]</code> target 5 actually returns 0 (correct) , because bug 1 masks bug 2 there. In isolation (<code>hi=len-1</code>), <code>[5]</code> target 5 gives -1	<code>[1, 3]</code> , 1	AI, by running the code	<code>lo <= hi</code>
3	Python	<code>lo = mid</code> never advances	Yes, hangs (killed by timeout, 11 of 34 exhaustive cases): <code>[1]</code> target 2, <code>[1, 3]</code> target 4, <code>[1, 3, 5, 7, 9]</code> target 4 (exit 124 under timeout 3). Note <code>[1, 3]</code> target 3 does NOT hang (returns 1)	<code>[1]</code> , 2	AI, by running with timeout	<code>lo = mid + 1</code>
4	C	<code>lo = mid</code> infinite loop	Yes, hangs with <code>n=5</code> , <code>a={1, 3, 5, 7, 9}</code> for targets 4, 9, 10 (exit 124). Targets 1, 5, 0 return correctly. The provided <code>test_binary_search.c</code> also hangs against the buggy file (exit 124) but passes against <code>binary_search_fixed.c</code>	<code>{1, 3, 5, 7, 9}</code> , 9	AI, by running with timeout	<code>lo = mid + 1</code>

#	Lang	Bug	Manifests?	Observed failing input	Found by	Fix
5	C	$(lo+hi)/2$ signed overflow	Not reproduced end-to-end: it needs an array of about 2^{31} elements. Only the arithmetic was demonstrated: with $lo=INT_MAX-1$, $hi=INT_MAX$ gcc -fsanitize=signed-integer-overflow reports "signed integer overflow" and $mid = -1$, versus correct 2147483646 with $lo+(hi-lo)/2$. Latent bug	arithmetic only	AI, by running a small overflow demo	$lo + (hi - lo)/2$
6	Java	$lo < hi$ misses the last candidate	Yes, wrong result: [5] target 5 gives -1; [1, 3] target 3 gives -1; [1, 3, 5, 7, 9] targets 3 and 9 give -1	{5}, 5	AI, by running a driver	$lo \leq hi$
7	Java	$(lo+hi)/2$ int overflow	Arithmetic demonstrated: $(MAX_VALUE - 1 + MAX_VALUE)/2$ prints -1 (correct 2147483646). Not reproduced with a real array (would need about 2^{31} elements)	arithmetic only	AI, by running a small demo	$lo + (hi - lo)/2$

Fixed versions: python/test_binary_search.py (5 tests OK), c/test_binary_search.c against binary_search_fixed.c ("All C tests passed"), java/BinarySearchTest ("All Java tests passed") all pass. Note the provided tests do not include a "target greater than all elements" case, so the hang in the buggy code is not caught by tests that only pass on the fixed version: it was found by the timeout runs above (test_binary_search.c did hang against the buggy C).

Source: [Ex06_AI_Assisted_Programming_Debugging/code_quality_analysis.md](#)

Code Quality Analysis - Manual vs AI-assisted

Measurement note. No stopwatch, no manual (human-only) implementation and no separate AI-only timing exist for this exercise. Metrics that need them are marked "not measured". Observations below come from running the code in this folder.

Metric	Manual coding	AI-assisted	Notes
Time to first working version (min)	not measured	not measured	No timings were recorded

Metric	Manual coding	AI-assisted	Notes
Bugs at first run	not measured	Buggy files: Python 3 planted bugs (2 observable: wrong result, hang; 1 masked), C 2 (1 hang, 1 latent overflow), Java 2 (1 wrong result, 1 latent overflow). Fixed files: 0 failures	See bugs.md. The buggy files are deliberately broken samples, not an AI's first attempt
Edge cases handled	not measured	Fixed versions handle empty array, single element hit/miss, first/last element, missing below/above range, 500k-element array (Python). Not tested: duplicates (returns some matching index, not necessarily the first), unsorted input, None, $C n < 0$, real overflow-sized arrays	
Test coverage	not measured	Python 5 tests, C 6 checks, Java 6 checks; no coverage tool was run so no percentage is claimed	Tests miss "target greater than all elements", which is exactly what triggers the hang in the buggy code
Readability / comments	not measured	Fixed versions are short and use the same structure across three languages; one-line docstring/Javadoc with complexity; C has no comment	
Complexity stated correctly	not measured	Yes: $O(\log n)$ time, $O(1)$ space iterative (matches the code)	See the section below
Languages done (Py/C/Java)	not measured	Python, C, Java all have buggy, fixed and test files, and all fixed versions pass	

Findings

Where AI helped most: Running the buggy code in three languages and exhaustively in Python (34 cases) showed exactly which bugs manifest and on which inputs, faster and more completely than reading the code.

Where AI (or the earlier notes) was wrong or overconfident: The earlier bugs.md claimed [5], target 5 fails in Python and that `hi = len(arr)` risks an index error. Running it showed both claims are false for the shipped file: bugs 1 and 2 mask each other on that input, and [1, 3] target 3 returns 1 correctly (the earlier table said it hangs). Reading code and asserting failures without running gave wrong claims. Overflow bugs (C and Java) could not be reproduced with a real array and are reported as latent.

What I would still do manually: Decide the intended semantics for duplicates (first or any match), review test adequacy (the tests miss the hang trigger), and run the overflow scenario on a machine with enough memory if it matters.

Complexity

Iterative binary search on a sorted array of n elements. Each iteration halves the remaining range $[lo, hi]$, so the loop runs at most $\text{floor}(\log_2 n) + 1$ times.

Case	Time	Why
Best	$O(1)$	Target is the middle element and is found on the first probe
Average	$O(\log n)$	About $\log_2(n) - 1$ probes for a successful search; about $\log_2(n)$ for an unsuccessful one
Worst	$O(\log n)$	Target absent or at the last remaining position: $T(n) = T(n/2) + O(1)$ gives $\log_2 n + 1$ probes (about 20 for 10^6 elements)
Space	$O(1)$	Only <code>lo</code> , <code>hi</code> , <code>mid</code> ; no recursion

The recursive form would use $O(\log n)$ stack space. The buggy `lo = mid` versions have no valid bound: they loop forever on some inputs.

Optimisation note

The fixed code is already asymptotically optimal for comparison-based search on a sorted array. Changes worth keeping: `lo + (hi - lo) // 2` avoids overflow in C and Java (in Python it is harmless because ints are unbounded); `<=` loop condition with `mid+1 / mid-1` guarantees progress. Possible micro-optimisations (not applied, not measured): use Python's `bisect.bisect_left` (C implementation, and it returns the leftmost match for duplicates); in C, a branchless variant or `size_t`/unsigned indexes for arrays above `INT_MAX`. For very few elements a linear scan can be faster due to branch prediction, but this was not benchmarked. Trade-off: extra variants reduce readability for gains that were not measured.

Source:

[Ex06_AI_Assisted_Programming_Debugging/prompts/prompts.md](#)

Prompt set (use once per language)

1. Generate

Write a `{{language}}` function `binary_search(arr, target)` returning the index or `-1` for a sorted array. Handle empty arrays and avoid integer overflow. Add comments.

2. Identify bugs

Find all bugs in this `{{language}}` code. For each: line, why it fails, failing input, fix. `{{code}}`

3. Optimise

Optimise this code for readability and performance without changing behaviour. Explain each change and any trade-off. `{{code}}`

4. Complexity

State the time and space complexity (best/avg/worst) of this code and justify with the recurrence or loop bound. `{{code}}`

5. Unit tests

Write unit tests for this function covering: empty array, single element (hit/miss), first/last element, duplicates, target smaller/larger than all, large array. Use `{{framework}}`. `{{code}}`

6. Compare manual vs AI

Given my manual solution and yours, list differences in correctness, edge cases, style and complexity.

Source:

[Ex06_AI_Assisted_Programming_Debugging/validation_run.md](#)

Browser compilation run: Experiment 6

Date: 2026-09-29 Online compiler: OneCompiler C and Java editors (onecompiler.com/c and onecompiler.com/java). Result: **All C tests passed** and **All Java tests passed**.

The local environment has no gcc, clang, MSVC cl, javac or java executable. To finish the checks in the browser, I placed each implementation with its equivalent bundled checks into a single editor source: six C checks and six Java checks. The browser compiler reported success for both. The repository's original separate-file build commands were not run locally, and compiler warnings were not recorded. Python's five-test suite had already passed locally.

Appendix: Source Code

Source:

Ex06_AI_Assisted_Programming_Debugging/c/binary_search_buggy.c

```
int binary_search(const int *arr, int n, int target) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2; /* BUG 1: (lo+hi) can overflow int for huge n */
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid; /* BUG 2: infinite loop, should be mid + 1 */
        else hi = mid - 1;
    }
    return -1;
}
```

Source:

Ex06_AI_Assisted_Programming_Debugging/c/binary_search_fixed.c

```
int binary_search(const int *arr, int n, int target) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Source:

Ex06_AI_Assisted_Programming_Debugging/c/test_binary_search.c

```
#include <stdio.h>
#include <stdlib.h>
int binary_search(const int *arr, int n, int target);

#define CHECK(cond) do { if (!(cond)) { printf("FAIL line %d: %s\n", __LINE__, #cond); return 1; } } while (0)

int main(void) {
    int a[] = {1, 3, 5, 7, 9};
    CHECK(binary_search(a, 0, 1) == -1);
    CHECK(binary_search(a, 5, 1) == 0);
    CHECK(binary_search(a, 5, 9) == 4);
    CHECK(binary_search(a, 5, 4) == -1);
    int one[] = {5};
    CHECK(binary_search(one, 1, 5) == 0);
    CHECK(binary_search(one, 1, 4) == -1);
    printf("All C tests passed\n");
    return 0;
}
```

Source:

Ex06_AI_Assisted_Programming_Debugging/java/BinarySearch.java

```
public class BinarySearch {
    /** Returns index of t in sorted array a, or -1. O(log n) time, O(1) space. */
    public static int search(int[] a, int t) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == t) return mid;
            if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}
```

Source: Ex06_AI_Assisted_Programming_Debugging/java/BinarySearchBuggy.java

```
public class BinarySearchBuggy {
    static int search(int[] a, int t) {
        int lo = 0, hi = a.length - 1;
        while (lo < hi) {                // BUG 1: should be lo <= hi (misses single remaining element)
            int mid = (lo + hi) / 2;      // BUG 2: overflow risk
            if (a[mid] == t) return mid;
            if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}
```

Source: Ex06_AI_Assisted_Programming_Debugging/java/BinarySearchTest.java

```
public class BinarySearchTest {
    static int failures = 0;
    static void eq(int exp, int got, String name) {
        if (exp != got) { failures++; System.out.println("FAIL " + name + ": expected " + exp + " got " + got); }
    }
    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7, 9};
        eq(-1, BinarySearch.search(new int[] {}, 1), "empty");
        eq(0, BinarySearch.search(a, 1), "first");
        eq(4, BinarySearch.search(a, 9), "last");
        eq(-1, BinarySearch.search(a, 4), "missing");
        eq(0, BinarySearch.search(new int[] {5}, 5), "single hit");
        eq(-1, BinarySearch.search(new int[] {5}, 4), "single miss");
        System.out.println(failures == 0 ? "All Java tests passed" : failures + " failure(s)");
        if (failures > 0) System.exit(1);
    }
}
```

Source: Ex06_AI_Assisted_Programming_Debugging/python/binary_search_buggy.py

```
def binary_search(arr, target):
    lo, hi = 0, len(arr)                # BUG 1: hi should be len(arr) - 1
    while lo < hi:                       # BUG 2: skips the last candidate with inclusive hi
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid                     # BUG 3: infinite loop, should be mid + 1
        else:
            hi = mid - 1
    return -1
```

Source: Ex06_AI_Assisted_Programming_Debugging/python/binary_search_fixed.py

```
def binary_search(arr, target):
    """Return index of target in sorted arr, or -1. O(log n) time, O(1) space."""
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Source: Ex06_AI_Assisted_Programming_Debugging/python/test_binary_search.py

```
import os, sys, unittest
sys.path.insert(0, os.path.dirname(__file__))
from binary_search_fixed import binary_search

class T(unittest.TestCase):
    def test_empty(self): self.assertEqual(binary_search([], 1), -1)
    def test_single(self):
        self.assertEqual(binary_search([5], 5), 0)
        self.assertEqual(binary_search([5], 4), -1)
    def test_ends(self):
        a = [1, 3, 5, 7, 9]
        self.assertEqual(binary_search(a, 1), 0)
        self.assertEqual(binary_search(a, 9), 4)
    def test_missing(self):
        a = [1, 3, 5]
        self.assertEqual(binary_search(a, 0), -1)
        self.assertEqual(binary_search(a, 6), -1)
    def test_large(self):
        a = list(range(0, 1_000_000, 2))
        self.assertEqual(binary_search(a, 999_998), 499_999)
        self.assertEqual(binary_search(a, 999_999), -1)

if __name__ == "__main__":
    unittest.main()
```